

DEEP LEARNING

Deep Learning for Natural Language Processing

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Table of Contents

1. Natural Language Processing

2. Representation Problem

3. Word2Vec

4. Transformer

What is NLP?

- Natural Language Processing = techniques that enable computers to process and generate human language.
- Example tasks:
 - ▶ Sentiment analysis (text $\rightarrow$ emotion)
 - ▶ Machine translation (en $\rightarrow$ fr)
 - ▶ Chatbots/question answering (question $\rightarrow$ answer)
 - ▶ Text summarization (long paragraph/document $\rightarrow$ summarized paragraph)
 - ▶ Named-entity recognition
- Each task has unique challenges but shares the need for good text representation.

Challenges in NLP

- Challenges in NLP:
 - ▶ Ambiguity: "bank" → financial institution versus riverbank
 - ▶ Long-range dependency: "The book that the professor who I met yesterday recommended was fascinating" (What is "fascinating"?)
 - ▶ World knowledge: "Paris is the capital of France."
- Humans effortlessly can perform these tasks, but it is challenging for a machine.
- Efforts for addressing these challenges:
 - ▶ Rule-based systems: expert-written grammars
 - ▶ Statistical NLP: n-grams, Hidden Markov Models, etc
 - ▶ Deep learning-based NLP: use deep neural networks (this lecture)

The Representation Problem

- Unlike humans, computers can not understand text directly – they need **numerical representations**.
- Q: How can we represent words in numerical values?
- Three different representations:
 - ▶ **One-hot encoding**: represent each word using a one-hot vector.
 - ▶ **Statistical representation**: represent a document by counting word occurrences
 - ▶ **Neural representation**: dense and continuous embeddings that are learned from data using neural networks

One-hot Representation

- Let's say we want to represent the words "cat" and "dog" numerically.
- Each word in a document gets a unique ID in a vocabulary of size V (the set of all possible words).
- With the **one-hot encoding** representation, we set the element corresponding to the word as 1 and all other elements as 0.
- Example:
 - ▶ Example for a vocabulary ($|V| = 4$): [dog, cat, apple, car]
 - ▶ "dog" $\rightarrow [1, 0, 0, 0]$
 - ▶ "cat" $\rightarrow [0, 1, 0, 0]$
- Problems:
 - ▶ No similarity: "dog" and "cat" are orthogonal
 - ▶ Huge dimensionality (millions of words)
 - ▶ No notion of context
 - ▶ Sparse: all but one element is 1

Statistical Representation

- **Bag of words** (BoW) representation: represent a document by counting word occurrences.
- Example:
 - ▶ Suppose the vocabulary is: [dog, cat, bone, eats]
 - ▶ "dog eats bone" $\rightarrow$ [1, 0, 1, 1]
 - ▶ "cat eats bone" $\rightarrow$ [0, 1, 1, 1]
- Problems:
 - ▶ Works well only for small-scale problems (eg, spam detection)
 - ▶ Sparse: most entries are 0 because most words don't appear in any single document
 - ▶ Orderless: "dog eats bone" and "bone eats dog" yields the same vector
 - ▶ No semantics: "dog" and "puppy" are completely unrelated in vector space
- TF-IDF is another statistical representation technique that weights words by their importance in the document versus corpus

Continuous Neural Representations

- Instead of representing a word as a one-hot vector, we represent it as a dense vector of real numbers.
- Example:
 - ▶ "dog" $\rightarrow$ [0.21, -0.43, 0.05, 0.88, ...]
 - ▶ "cat" $\rightarrow$ [0.19, -0.40, 0.07, 0.90, ...]
- These are also called **word embeddings**, which is a vector of 50-300 dimensions (in early methods) or even much higher (in modern day LLMs).
- It is based on the intuition: "Words that occur in similar contexts tend to have similar meanings."
- Two popular ways of learning word representations:
 - ▶ Early neural embedding models like **word2vec**
 - ▶ Modern day **Transformer**-based LLMs

Word2Vec

"dog" $\rightarrow$ [0.21, -0.43, 0.05, 0.88, $\dots$, -0.01]

Word2Vec Representation

- Word2Vec is a method that creates a fixed-length vector representation of each word using a neural network. It was proposed by Mikholov et. al in 2013 (researches from Google).
- Words that are semantically similar will be closer in the representation space¹.



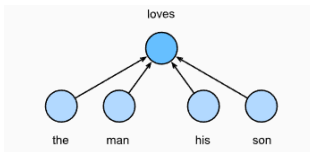
¹Supporting notebook

Word2Vec Representation

- There are two main approaches for implementing Word2Vec:
 - ▶ Continuous Bag-of-Words or CBOW (we will only explore this technique)
 - ▶ Skip-gram
- CBOW is a form of **unsupervised learning** using a shallow neural network that does not require explicit supervision or human annotations.
- All we need to do is to collect a lot of data from the internet or some known corpus, which is large enough.
- This kind of learning became very popular in the later years: both in NLP and Computer Vision.

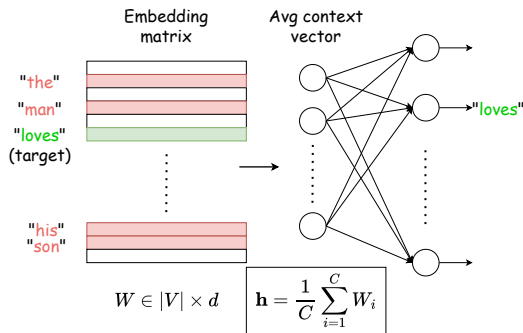
Continuous Bag-of-Words

- The CBOW model assumes that a center word is generated based on its surrounding context words in a text sequence.
- For example, consider the text sequence "the", "man", "loves", "his", "son".
- Given the context words "the", "man" (on the left), and "his", "son" (on the right), the model must predict the center word "loves".



- Note, how this model can be trained without having to annotate the ground truth (or target) value, simply by collecting a corpus of text and artificially creating input/output pairs:
 - ▶ Input: "the", "man", "his", "son"
 - ▶ Output: "loves"

Continuous Bag-of-Words



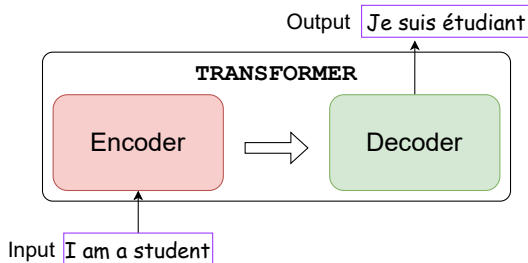
- We have a set of context words (in red) and a target word (in green).
- Each context word is mapped to its embedding vector $W_i \in \mathbb{R}^d$ (a **learnable embedding**). For a vocabulary of size $|V|$, we get a matrix $W \in \mathbb{R}^{|V| \times d}$.
- Given the context words, an average context vector (or embedding) is computed.
- Apply a linear layer (or output layer) to predict the target word's probability distribution (via softmax). Trained using Cross Entropy Loss + SGD.

Continuous Bag-of-Words

- Once the network is trained, we cut away the output layer.
- We are left with an embedding matrix $W \in \mathbb{R}^{|V| \times d}$ with the values learned through SGD.
- In W , the i -th row corresponds to the embedding or (representation) of the i -th word in the vocabulary. This is what we called as the word2vec representation.
- Obtaining the vector representation for a word is simply a lookup operation given the index of the word.
- Major issues of word2vec representation:
 - ▶ Static embedding size
 - ▶ No representation for out-of-vocabulary words, ie, words not seen before
 - ▶ Word orders are ignored (due to averaging), that is why its a bag-of-words representation

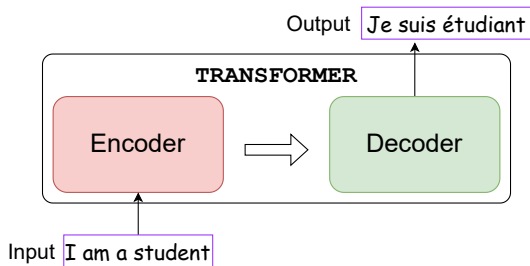
Transformers

Transformer



Existing models (eg, RNN)	Transformer
Struggled over long context Sequential computation Ordering was implicit Hard to scale	Attend to all words at once (self-attention) Trained faster by processing sequences in parallel Kept word order using positional encodings Scaled effortlessly

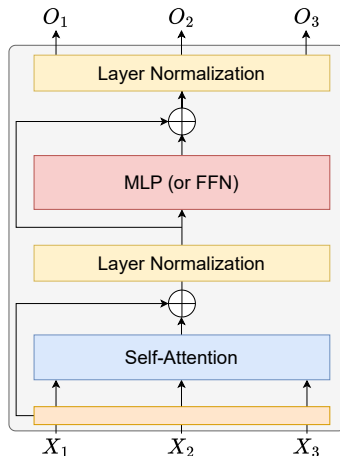
Transformer Architecture



- Transformer is comprised of 2 main building blocks:
 - ▶ Encoder
 - ▶ Decoder

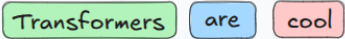
Transformer Block

- A **transformer block** takes as input a set of vectors, and outputs a set of vector, of the same dimensions.
- It consists of the following components:
 - ▶ Attention (has several variants: self-attention, cross-attention, masked self-attention and multi-head attention)
 - ▶ Layer Normalization
 - ▶ MLP (or FFN)
 - ▶ Residual Connections
 - ▶ Positional Encoding (plus the input embeddings that convert word to numbers)
- The transformer block is repeated several times in the whole architecture.



A detour: Converting Words into Numbers

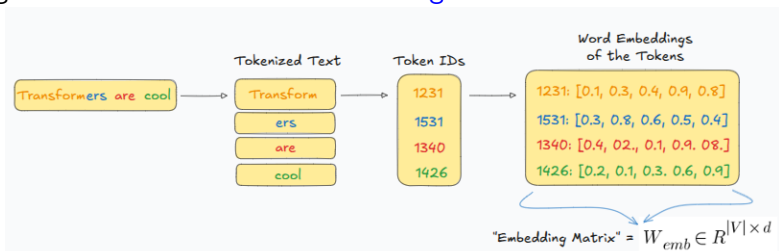
- As a neural network can only understand numbers, we first need to convert our sentences (or words) into a numerical representation. The first step of this process involves breaking down a sentence into smaller units called **tokens**, and the process is called **tokenization**².
- Let's say our sentence is: "Transformers are cool"

Type of Tokenization	Example
Word	
Subword	
Character	

²Tiktokenizer demo

A detour: Converting Words into Numbers

- There are various algorithms for tokenization, and one popular approach is called Byte Pair Encoding (BPE)³.
- Each unique token is mapped to a unique token ID, which are integer numbers.
- Since NNs are not good with purely integer values, we need a mechanism to convert these integers into vectors or **token embeddings**.



- Just as in word2vec, each row in the embedding matrix is a trainable vector representing a token.

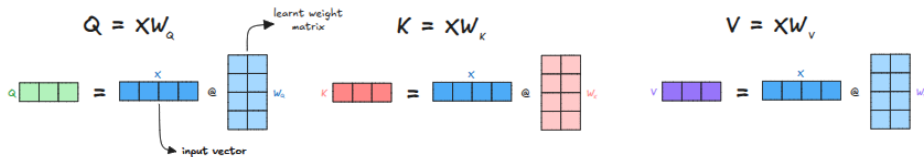
³BPE tutorial

Self-Attention

- Attention relies on 3 key inputs for each token
 - ▶ A **Query Vector** (Q): what the token wants to find out
 - ▶ A **Key Vector** (K): what this token offers to others
 - ▶ A **Value Vector** (V): the actual content this token can contribute to
- As an intuitive example, think of Google search:
 - ▶ What you type in the search bar is **query vector** (Q)
 - ▶ What the search engine compares your query with (eg, the titles or keywords in webpages) is **key vector** (K)
 - ▶ The actual content of the webpage is the **value vector** (V)

Self-Attention

- In self-attention, the Q , K and V are all derived from the same input matrix.



- The input vector (X) is transformed into query, key and value vectors by multiplying with the weight matrix (W) for each of the W_Q , W_K and W_V matrices.

$$Q = XW_Q, K = XW_K, V = XW_V$$

- The weight matrices: W_Q , W_K and W_V are learned via backpropagation.

Self-Attention

- The attention weights are computed using the formula: $A = \text{softmax}(\frac{QK^T}{\sqrt{d}})$
- Softmax ensures that the contribution of each query vector for every key vector sums to 1. And we divide by $\sqrt{d}$, ie, the dimension of the vector, for numerical stability.

	He	swung	the	bat	with	incredible	force
He	0.35	0.10	0.05	0.05	0.05	0.05	0.35
swung	0.10	0.25	0.10	0.25	0.10	0.10	0.15
the	0.05	0.10	0.40	0.25	0.10	0.05	0.05
bat	0.05	0.20	0.10	0.30	0.15	0.10	0.10
with	0.05	0.10	0.10	0.15	0.30	0.20	0.10
incredible	0.05	0.10	0.05	0.10	0.25	0.35	0.10
force	0.35	0.05	0.05	0.05	0.05	0.10	0.35

sum along rows = 1

Self-Attention

- Last step in self-attention operation is to multiply the attention scores by the value matrix:

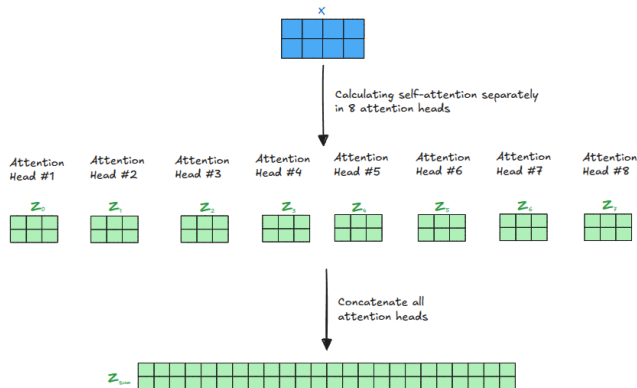
$$SA(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

- **Intuitively**, the output of the above operation becomes a new representation that captures both the meaning of each word and its context within the sentence.

Multi-Head Attention

- Let's consider the given sentence: "He swung the bat with incredible force"
- We may need to capture two relationships
 - ▶ "swung" → "bat" (the action and the object)
 - ▶ "incredible" → "force" (description and intensity)
- Multi-headed attention lets the model look at all those relationships in parallel.
- Each head is in charge of paying attention which is slightly different from the other.
- For example: one head may focus on grammar, while the other on emotion and so on.
- From a computational perspective, a head is nothing but independent SA operations, but computed in parallel, independent of each other.

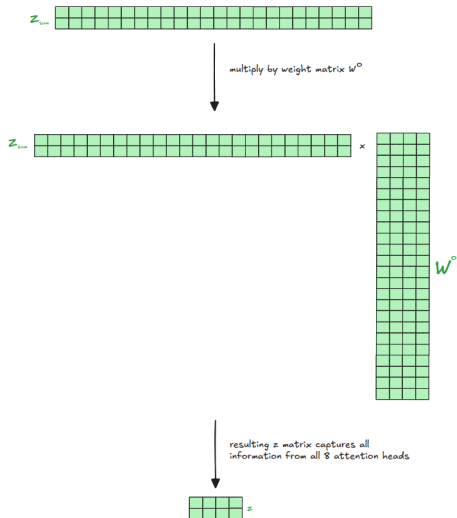
Multi-Head Attention



- In this example, for a given input matrix X , we are calculating the outputs from eight different self-attention operations, followed by concatenation.
- Note that the weight matrices W are reused for all the attention heads, ie, not one W per attention head.

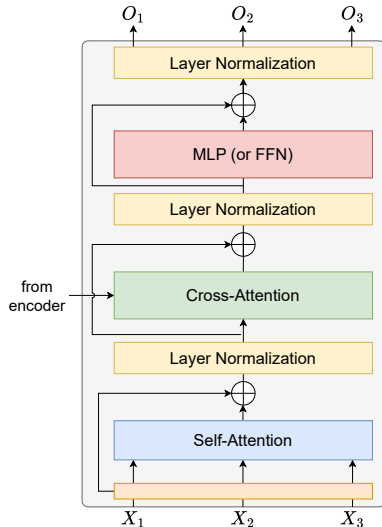
Multi-Head Attention

- After concatenation, we multiply the concatenated matrix with a final projection matrix W_O .
- The W_O blends the insights from every head into a single, context-aware representation.



Transformer Decoder

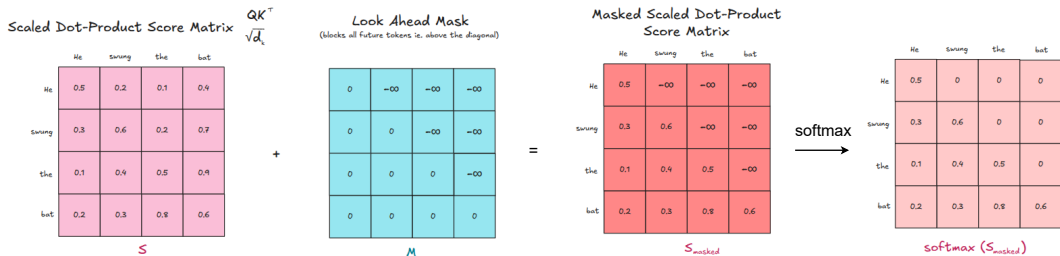
- The transformer block in the decoder is almost identical to that of the encoder, with two differences.
- First, it has a cross-attention block, where it receives an additional input from the encoder.
- Second, the self-attention layer has **masked attention**, which prevents vectors to “look ahead” in the output sequence.



Masked Self-Attention

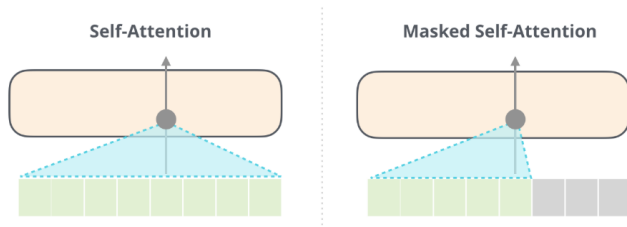
- In the decoder, the first attention layer is slightly modified, it's called: masked self-attention.
- In regular self-attention, every token can "see" every other token in the sequence, which works perfectly for the encoder (since the entire input is known).
- Here, the same self-attention mechanism applies, but with a twist: the model is forbidden from looking at **future tokens** when predicting the next word.
- Consider an example: "He swung the bat".
- While predicting the word "swung", the network should not look at the successive word "the bat".
- The model is allowed to only look at the word "He" while predicting "swung".
- Masked self-attention creates a **look-ahead mask** that prevents the model from looking-ahead in the sentence.

Masked Self-Attention



- If we did not have this masking operation, then the model would already know the future tokens in advance.
- Instead of actually learning to predict the next word (or token), it will simply copy information from the future tokens.

Self-Attention versus Masked Self-Attention



- **Masked self-attention** is a mechanism that restricts which tokens a model can attend to during training or inference.
- Let's say the model has to predict the next word "buy" for this input sentence: "I went to the store to buy some milk" (Gray means not seen so far).
- If self-attention can see the entire sentence, then it can cheat by looking at "milk".
- Masked self-attention hides away the future segment (the one in gray). Now the model must infer "buy" based only on prior context.

Cross-Attention

- In the case of language translation task (eg, en $\rightarrow$ fr), the model still needs a way to connect back to the input sentence (in en) the encoder processed.
- Cross-attention allows the decoder to **look over** at the encoder's output to decide which parts of the input matter the most for generating the next word.

target sequence: il a balancé la batte avec une force incroyable <EOS>

source sequence: he swung the bat with incredible force <EOS>

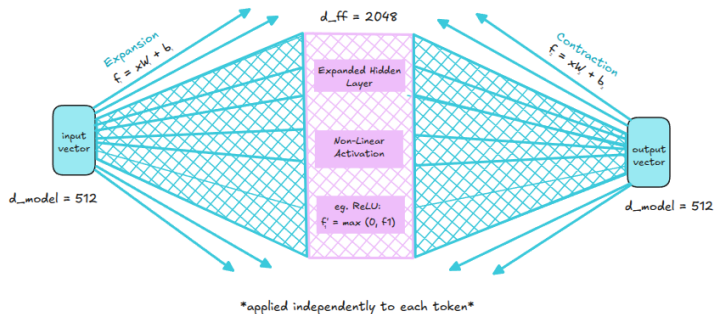
input to the encoder: he swung the bat with incredible force <EOS>

input to the decoder: <start> il a balancé la batte avec une force incroyable <EOS>

- The encoder's output provides the key (K) and value (V) to the decoder's second multi-head attention layer.
- The query (Q) comes from the decoder's own hidden states (based on what's been generated so far).

Feed-Forward Neural Networks

- Once attention has done its job capturing relationships between tokens and layering in contextual meaning, the Transformer passes that information through a Feed-Forward Neural Network (FFN).
- While attention tells the model what to focus on, the FFN decides what to do with that information.
- The FFN is exactly like a MLP we have seen in the earlier lectures.



Layer Normalization

- A transformer has billions of parameters, and gradients can easily explode or vanish as signals move through layers.
- To prevent instability during training, we use normalization inside a transformer block, called **Layer Normalization**. This normalizes the features to have zero mean and unit standard deviation.

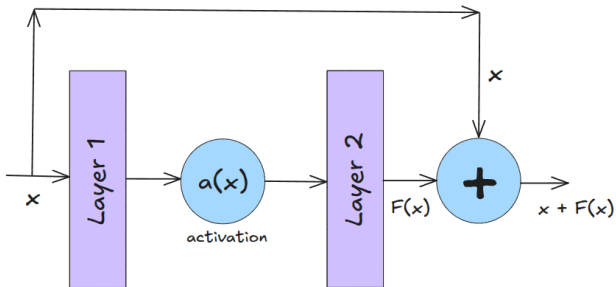
The diagram illustrates the Layer Normalization formula with the following components and annotations:

- Scaling parameter (γ)**: Labeled "scaling parameter (learnt during training)".
- Input (x)**: Labeled "1 of the outputs from the sublayer".
- Mean (μ)**: Labeled "mean of all the d outputs of the sublayer".
- Variance (σ^2)**: Labeled "variance between all the outputs of the sublayer".
- Small positive constant (ϵ)**: Labeled "small positive constant added for numeric stability (to avoid dividing by 0)".
- Shifting parameter (β)**: Labeled "shifting parameter (learnt during training)".

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

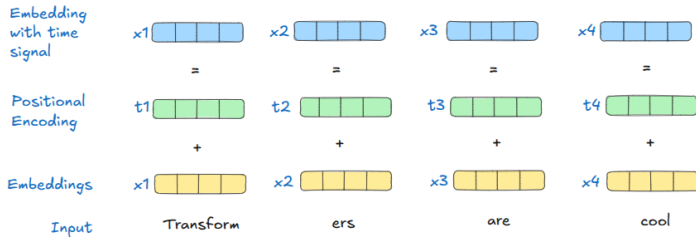
Skip Connections

- Just as in very deep convolutional neural networks, transformers are also vulnerable to vanishing gradient, especially because of several hundreds of layers.
- To solve this, we use **skip connections**, as in ResNets.



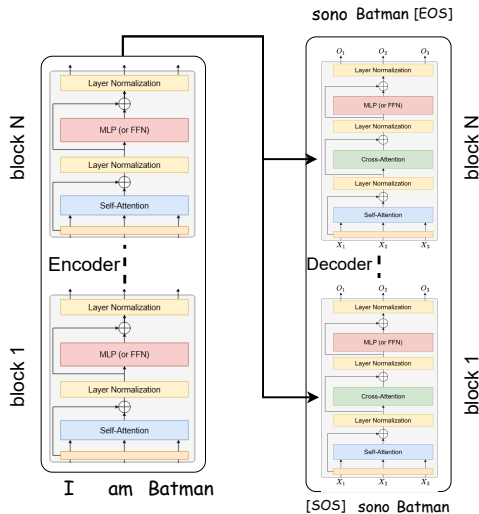
Positional Encoding

- Unlike RNNs, which process tokens sequentially, Transformers see the entire input at once. That parallelism is what makes them so fast...but it also means they lose any built-in **sense of order**.
- To fix this, we introduce positional encodings, which is nothing but a vector that is conditioned on the position in the sentence.

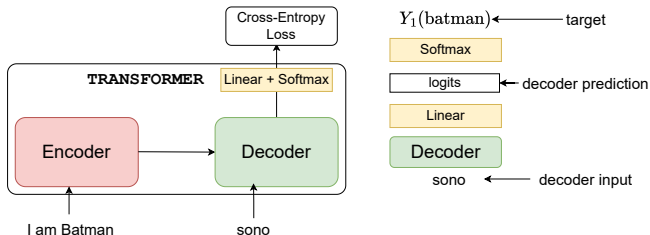


Transformer: Full Architecture

- This is the entire Transformer architecture with Encoder and Decoder put together.
- [SOS] and [EOS] are special words (or tokens) that mark the start and end of a sentence. Both the encoder and decoder have them (omitted for simplicity).

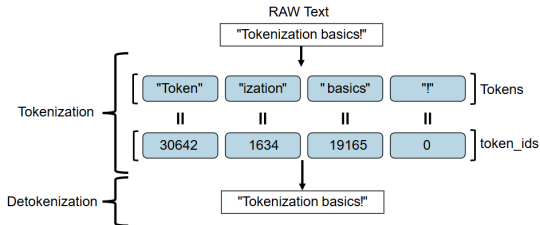


Transformer Training



- We feed the entire source sentence (in en) to the encoder and only a part of the target sentence (in fr) to the decoder. The model is trained to predict the next token in the target sentence.
- Cross-entropy loss or KL Divergence is used for training Transformers.
- In the above example, for the input “sono”, the decoder must predict the next token “batman” (or the index corresponding to “batman” in the vocabulary).

Detokenization at the Output



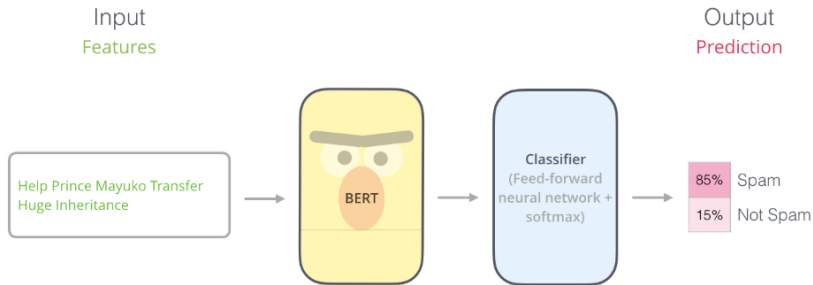
- At the output of the decoder, the model predicts the most likely token ID.
- The token ID at the output is then converted to the corresponding raw text to get the model's prediction, through [detokenization](#).

Various Flavours of Transformers

- The original Transformer paper used the encoder-decoder architecture for the task of machine translation, ie, for translating from one language to the other.
- However, this is not the only way to use transformers. There are a couple of other variants:
 - ▶ Encoder-only transformer, where only the encoder is present (eg, BERT, ELMo models from Google)
 - ▶ Decoder-only transformer, where only the decoder is present (eg, GPT, GPT-2, ChatGPT models from Open AI)
- Encoder-only transformers are mainly used for classification tasks, such as sentiment classification.
- Decoder-only transformers are mainly used for generating text, such as writing an essay.

Encoder-only Transformer

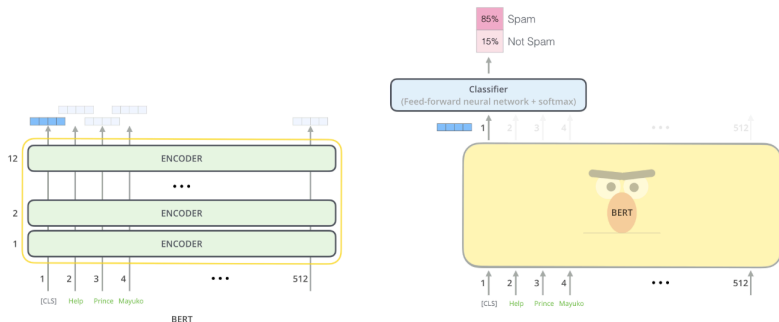
- BERT, which stands for Bidirectional Encoder Representations from Transformers is an encoder-only transformer⁴.
- It is used for classification tasks such as sentence classification, sentiment classification and so on.



⁴BERT tutorial

BERT

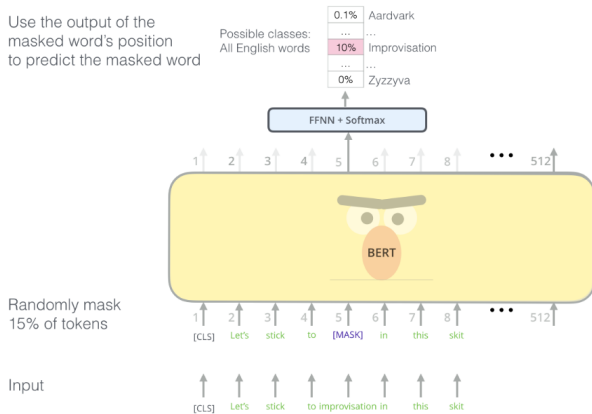
- At its core, BERT is a stack of transformer encoder blocks. It has no decoder blocks.



- We add a special token called [CLS], which acts as an aggregate representation of the entire sentence (thanks to self-attention).
- After training, we simply add a linear classifier on top of the [CLS] token to address any downstream classification task.

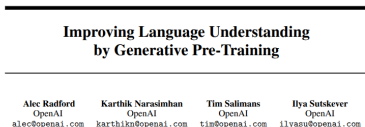
Training BERT

- There are a couple of ways to train BERT:
 - ▶ Masked Language Modelling (we will only cover this)
 - ▶ Two-sentences Task



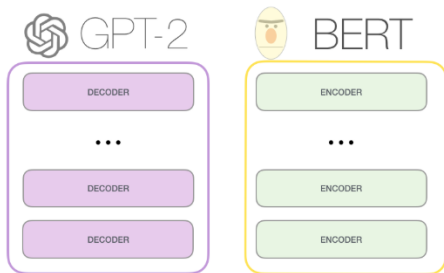
Decoder-only Transformer

- Decoder-only transformers only consist of decoder block and has no encoder blocks, as in the original transformer paper.
- Decoder-only transformers are also called **autoregressive models** or **generative pretrained transformer** (GPT) due to their ability to generate new text.

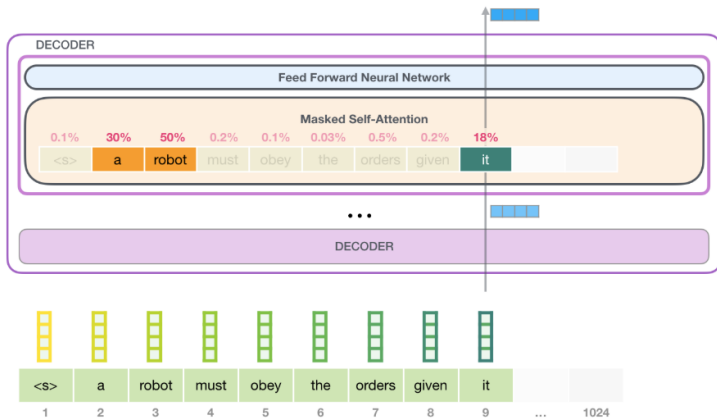


Abstract

Natural language understanding comprises a wide range of diverse tasks such as textual entailment, question answering, semantic similarity assessment, and document classification. Although large unlabeled text corpora are abundant, labeled data for learning these specific tasks is scarce, making it challenging for discriminatively trained models to perform adequately. We demonstrate that large gains on these tasks can be realized by *generative pre-training* of a language model on a diverse corpus of unlabeled text, followed by *discriminative fine-tuning* on each specific task. In contrast to previous approaches, we make use of task-aware input transformations during fine-tuning to achieve effective transfer while requiring minimal changes to the model architecture. We demonstrate the effectiveness of our approach on a wide range of benchmarks for natural language understanding. Our general task-agnostic model outperforms discriminatively trained models that use architectures specifically crafted for each task, significantly improving upon the state of the art in 9 out of the 12 tasks studied. For instance, we achieve absolute improvements of 8.9% on commonsense reasoning (Stories Cloze Test), 5.7% on question answering (RACE), and 1.5% on textual entailment (MultiNLI).



Training GPT



- GPT is trained by predicting the next word in a sequence using large amounts of text, learning language patterns through unsupervised pre-training.

Closing Remarks

- Original Transformer:
 - ▶ Introduced the attention mechanism — “Attention is all you need.”
 - ▶ Uses encoder–decoder architecture for sequence-to-sequence tasks (e.g., translation).
- Encoder-Only Transformers:
 - ▶ Focus on understanding text (e.g., BERT).
 - ▶ Use bidirectional context to build deep language representations.
- Decoder-Only Transformers:
 - ▶ Focus on generating text (e.g., GPT).
 - ▶ Use left-to-right context for next-word prediction.

References and Links

- For additional reading materials refer to the following links
 - ▶ [“The Illustrated Transformer”](#)⁵.
 - ▶ [“Attention? Attention!”](#)⁶.
 - ▶ [“The Illustrated GPT-2”](#)⁷.
 - ▶ [“Everything About Transformers”](#)⁸.
 - ▶ [“Decoder-Only Transformers”](#)⁹.
- Some slides have been adapted from J. Johnson.

⁵<https://jalammar.github.io/illustrated-transformer/>

⁶<https://lilianweng.github.io/posts/2018-06-24-attention/>

⁷<https://jalammar.github.io/illustrated-gpt2/>

⁸<https://www.krupadave.com/articles/everything-about-transformers>

⁹<https://cameronrwolfe.substack.com/p/decoder-only-transformers-the-workhorse>